



Introduzione a vue.js

Programmazione

Vue.js è un framework JavaScript progressivo utilizzato per creare interfacce utente moderne, dinamiche e reattive.

Viene definito “progressivo” perché può essere adottato gradualmente: può servire sia per aggiungere piccole funzionalità interattive a una pagina esistente, sia per costruire applicazioni complete come Single Page Application, dashboard, gestionali o frontend complessi.

Vue si basa su un approccio component-based, quindi l'interfaccia viene suddivisa in componenti riutilizzabili, ognuno con la propria struttura, logica e stile.



Dal punto di vista tecnico, Vue combina una sintassi semplice e leggibile con un sistema di reattività molto potente.

Quando i dati dell'applicazione cambiano, Vue aggiorna automaticamente la parte dell'interfaccia coinvolta, riducendo la necessità di manipolare manualmente il DOM.

I concetti principali ruotano attorno a componenti, template, data binding, direttive, eventi, props, state locale e lifecycle del componente.

Questa struttura rende Vue è adatto sia a chi si avvicina per la prima volta ai framework frontend, sia a progetti professionali che richiedono codice modulare, mantenibile e scalabile.



4 caratteristiche principali di Vue

- **Framework progressivo**

Vue può essere introdotto gradualmente in un progetto. Può essere usato per rendere interattiva una singola parte di una pagina HTML, oppure per costruire applicazioni complete e strutturate. Questo lo rende flessibile sia per piccoli progetti sia per frontend complessi.

- **Component-based**

Vue permette di dividere l'interfaccia in componenti riutilizzabili. Ogni componente può contenere template, logica JavaScript e stile CSS, rendendo il codice più ordinato, modulare e facile da mantenere.



4 caratteristiche principali di Vue

- **Reattività dei dati**

Vue aggiorna automaticamente l'interfaccia quando cambiano i dati dell'applicazione. Lo sviluppatore definisce i dati e il modo in cui devono essere mostrati, mentre Vue si occupa di sincronizzare la UI con lo stato interno.

- **Sintassi semplice e leggibile**

Vue utilizza una sintassi vicina a HTML, CSS e JavaScript tradizionali. Grazie a template, direttive come v-if, v-for, v-model e gestione eventi con v-on, risulta più immediato da apprendere rispetto ad altri framework frontend molto strutturati.



Configurazione iniziale di Vue

Per configurare un progetto Vue moderno si utilizza normalmente Node.js insieme a Vite, uno strumento che permette di creare rapidamente la struttura del progetto, avviare un server locale e preparare l'applicazione per la pubblicazione.

Vue può essere usato anche tramite CDN in una semplice pagina HTML, ma per un progetto reale è preferibile usare una struttura completa con componenti .vue.

- 1. `npm create vue@latest mio-progetto-vue`**
- 2. `cd mio-progetto-vue`**
- 3. `npm install`**
- 4. `npm run dev`**



Componenti

I componenti sono i blocchi principali di un'applicazione Vue. Ogni componente può rappresentare una parte dell'interfaccia, come un menu, una card, un form o una pagina.

Nei file .vue un componente può contenere template HTML, logica JavaScript e stile CSS nello stesso file, rendendo il codice organizzato e riutilizzabile.

```

1.<template>
2.  <div>
3.    <h1>Benvenuto in Vue</h1>
4.    <p>Questo è il mio primo componente.</p>
5.  </div>
6.</template>
7.
8.<script>
9.export default {
10.  name: "Benvenuto"
11.};
12.</script>
    
```



Data binding

Il data binding permette di collegare i dati JavaScript alla parte visiva dell'interfaccia.

Quando il valore di una variabile cambia, Vue aggiorna automaticamente il contenuto mostrato nella pagina, questo evita di modificare manualmente il DOM e rende tutto dinamico.

```
1.<template>
2. <div>
3.   <h2>{{ titolo }}</h2>
4.   <p>{{ descrizione }}</p>
5. </div>
6.</template>
7.
8.<script>
9.export default {
10. data() {
11.   return {
12.     titolo: "Corso Vue",
13.     descrizione: "Introduzione al framework Vue.js"
14.   };
15. }
16.};
17.</script>
```



Direttive Vue

Le direttive sono attributi speciali usati nei template per controllare il comportamento degli elementi HTML.

Tra le più comuni ci sono v-if per mostrare elementi in base a una condizione, v-for per generare liste e v-bind per collegare attributi HTML a variabili JavaScript.

```

1.<template>
2. <div>
3.   <p v-if="utenteLoggato">Benvenuto utente!</p>
4.   <p v-else>Effettua il login.</p>
5.
6.   <ul>
7.     <li v-for="corso in corsi" :key="corso">
8.       {{ corso }}
9.     </li>
10.  </ul>
11. </div>
12.</template>
13.
14.<script>
15.export default {
16.  data() {
17.    return {
18.      utenteLoggato: true,
19.      corsi: ["Vue", "React", "Angular"]
20.    };
21.  }
22.};
23.</script>

```



Eventi

Vue permette di gestire gli eventi dell'utente, come click, input, submit o selezioni.

Gli eventi vengono collegati agli elementi HTML tramite v-on, oppure con la sintassi abbreviata @. Questo consente di eseguire funzioni quando l'utente interagisce con l'interfaccia.

```

1.<template>
2. <div>
3.   <p>Valore: {{ contatore }}</p>
4.
5.   <button @click="aumenta">
6.     Aumenta
7.   </button>
8. </div>
9.</template>
10.
11.<script>
12.export default {
13.  data() {
14.    return {
15.      contatore: 0
16.    };
17.  },
18.  methods: {
19.    aumenta() {
20.      this.contatore++;
21.    }
22.  }
23.};
24.</script>

```



Props

Le props permettono di passare dati da un componente padre a un componente figlio.

Sono utili per creare componenti riutilizzabili, perché lo stesso componente può mostrare contenuti diversi in base ai valori ricevuti dall'esterno.

```

1.<!-- CardCorso.vue -->
2.<template>
3.  <div>
4.    <h2>{{ titolo }}</h2>
5.    <p>{{ descrizione }}</p>
6.  </div>
7.</template>
8.
9.<script>
10.export default {
11.  props: {
12.    titolo: String,
13.    descrizione: String
14.  }
15.};
16.</script>

```



Form e v-model

v-model permette di collegare un campo di input a una variabile del componente.

Questo meccanismo è chiamato two-way data binding, perché il dato aggiorna l'interfaccia e l'interfaccia aggiorna il dato.

È molto usato nella gestione di form, filtri, ricerche e campi modificabili dall'utente.

```
1.<template>
2. <div>
3.   <input
4.     v-model="nome"
5.     placeholder="Inserisci il tuo nome"
6.   />
7.
8.   <p>Nome inserito: {{ nome }}</p>
9. </div>
10.</template>
11.
12.<script>
13.export default {
14.  data() {
15.    return {
16.      nome: ""
17.    };
18.  }
19.};
20.</script>
```



Le problematiche comuni nello sviluppo con Vue riguardano soprattutto l'organizzazione dei componenti, la gestione dello stato e l'uso corretto della reattività.

Nei progetti piccoli Vue risulta molto semplice e immediato, ma quando l'applicazione cresce possono emergere componenti troppo grandi, logica duplicata, props passate attraverso troppi livelli, eventi difficili da tracciare e una struttura poco chiara.

Un altro errore frequente è usare v-if, v-for, watch, computed e v-model senza una progettazione precisa, rendendo il codice meno leggibile e più difficile da mantenere.

Inoltre, Vue da solo non copre sempre tutte le esigenze di un'applicazione completa: per routing, gestione globale dello stato, validazione form, autenticazione e chiamate API strutturate servono spesso strumenti aggiuntivi come Vue Router, Pinia o librerie dedicate.



Buon Davante a tutti

